staunton

Chess diagrams, games and tournament tables for Typst

User manual · package version 0.1.0

Contents

1 Introduction

staunton is a Typst package for chess publications. From a FEN string, a parsed PGN, or a position you build by hand, it produces:

- **boards and diagrams** — pure boards with labels, highlights, arrows, an optional grid, flexible sizing, custom colours, and bundled SVG piece sets (or a Unicode fallback); and building on that diagrams with captions, figure counters, and referenceable labels;
- **games from PGN** — a sophisticated parser creates single games or an array of games, from which you create positions by "locators" (mainline and variations), move play-out, and FEN export;
- **move notation** — move text output with localized piece letters, figurine glyphs, NAGs, comments and diagrams embedded inline;
- **tournament tables** — standings, cross-tables and progress charts from a PGN's results, by player or by team;
- **outlines and references** — diagrams and tables get their own counters and lists;
- **document-wide styling** and **localization** (six languages, easily extended).

```
#chess-diagram(
  "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/
PPPP1PPP/RNBQKB1R",
  caption: [The Ruy Lopez, three moves
in.],
  size: 4cm,
)
```

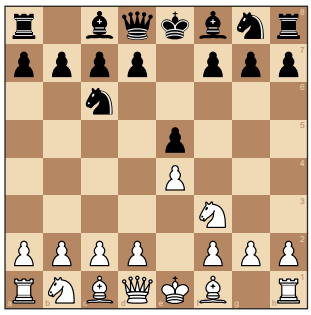


Diagram 1: The Ruy Lopez, three moves in.

This manual is bottom-up. The **board** is the drawing primitive; a **diagram** wraps a board in a referenceable figure; a **position** is the data a board draws; **games** add PGN, notation, and play; then come **tournament tables**, **outlines and references**, and the **document-wide style** settings.

1.1 The name

Typst package **staunton** is named after **Howard Staunton** (c. 1810–1874): a leading chess master of his day, organiser of the first international tournament (London, 1851), a chess author and publisher, and the namesake of the standardised **Staunton pattern** chessmen — still the tournament standard.

1.2 Installing and importing

staunton is a Typst package. Import its public API once and every function in this manual is in scope:

```
#import "@preview/staunton:0.1.0": *
```

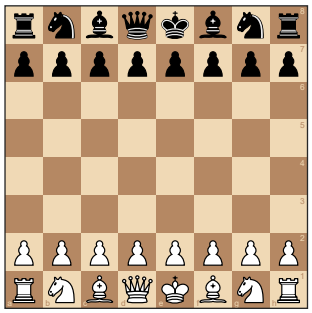
1.3 How to read this manual

In every framed example, the left side is **the code you type** and the right side is **exactly what it renders** — the manual compiles its own examples, so the two can never disagree.

2 The board

`board(source, ..)` draws **just the board** — no caption, no figure — and is the primitive every diagram builds on. `source` is one of: a **FEN string**; a **position** (from `position(..)` or `parse-fen(..)`); or a bare **squares** dict (square name ⟶ `(kind, color)`).

```
#board(
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/
RNBQKBNR",
  size: 4cm,
)
```



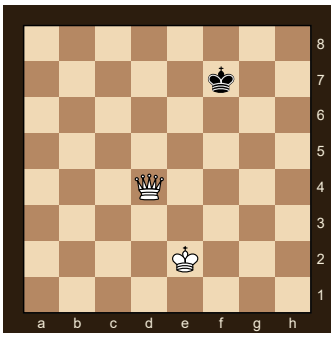
`board` is the variant-agnostic primitive; `chess-board` is the standard-chess sugar over it (same rendering, but it documents the variant and rejects a non-standard source). Use `board` inline in text or inside your own layout; reach for a **diagram** (next chapter) when you want a captioned, referenceable figure.

The rest of this chapter covers the board's drawing options: labels, highlights, arrows, the grid, coordinates, size, colours, orientation, and piece sets — all of which a `chess-diagram` accepts too.

2.1 Labels

`label-mode` chooses how files and ranks are drawn — `"on-square"` (default, tucked into the corner squares), `"outside"` (a gutter strip), or `"border"` (a themed band, styled by `border-theme`). `labels: false` suppresses them.

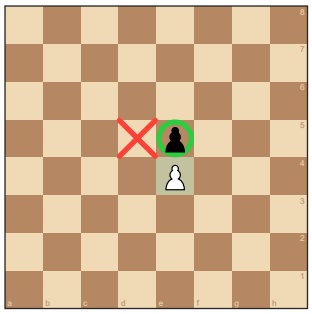
```
#board(
  "8/5k2/8/8/3Q4/8/4K3/8",
  label-mode: "border",
  border-theme: "brown",
  size: 3.8cm,
)
```



2.2 Highlights

`highlight` marks squares; each entry is a square name (drawn with `highlight-shape`, default `"filled"`), a `(square, color)` pair, or a dict `(square: .., shape: .., color: ..)` where `shape` is `"filled"`, `"cross"`, or `"circle"`. By convention a **cross** marks an empty square.

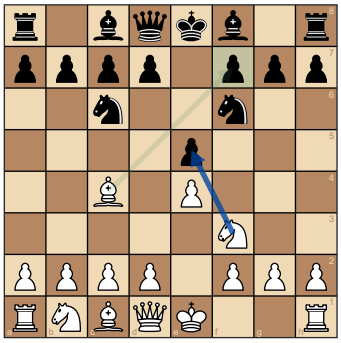
```
#board(
  "8/8/8/4p3/4P3/8/8/8",
  highlight: (
    "e4",
    (square: "e5", shape: "circle"),
    (square: "d5", shape: "cross"),
  ),
  size: 4cm,
)
```



2.3 Arrows and the grid

`arrows` draws arrows; each entry is a `(from, to)` or `(from, to, color)` tuple, or a dict `(from: .., to: .., color: ..)`. A missing color uses `arrow-color`. Arrows scale with the board and flip with it. A `grid: true` overlay draws thin lines between the squares.

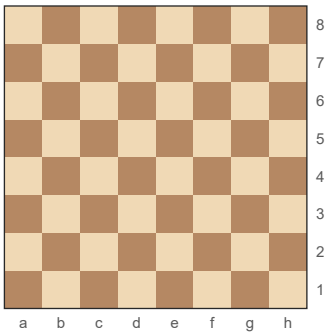
```
#board(
  "r1bqkb1r/pppp1ppp/2n2n2/4p3/2B1P3/5N2/PPPP1PPP/RNBQK2R",
  grid: true,
  highlight: ("f7",),
  arrows: (("c4", "f7"), ("f3", "e5", rgb(0, 70, 160, 200))),
  size: 4.4cm,
)
```



2.4 Coordinates and non-square boards

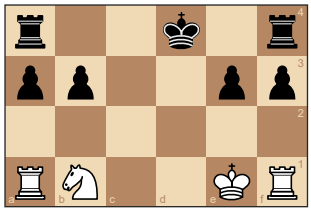
Files run `a`, `b`, … and ranks `1`, `2`, …; `a1` is the dark square in the lower-left corner, `h8` the upper-right. Square names are case-insensitive (`"E4"` = `"e4"`).

```
#board(
  "8/8/8/8/8/8/8/8",
  label-mode: "outside",
  size: 4cm,
)
```



The board is **not** tied to 8×8. A `position` built from the string form (next chapter) counts its own columns and rows, and the renderer draws whatever geometry it is given — files and ranks extend as far as the board needs, and the cells stay square while the board itself becomes rectangular:

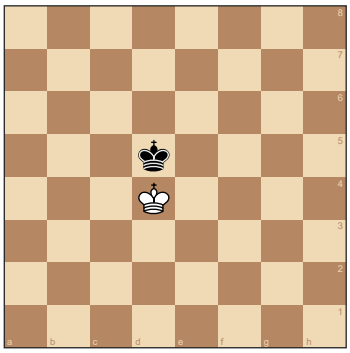
```
#board(
  position(
    "r..k.r",
    "pp..pp",
    "......",
    "RN..KR",
  ),
  size: 4cm,
)
```



2.5 Sizing

The default board size suits an A4 two-column layout. A board reads the available space at its insertion point via `layout`, so it adapts to any column or page size without being told the geometry, and shrinks to fit if asked for more than fits. `size` may be a `length`, a `ratio` of the available width, or `auto`:

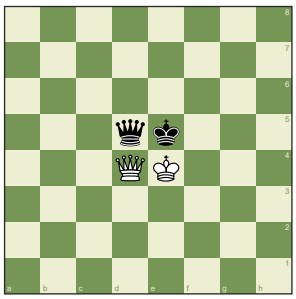
```
#board(
  "8/8/8/3k4/3K4/8/8/8",
  size: 60%,   // of the available
width
)
```



2.6 Colours

`light` and `dark` set the two square colours:

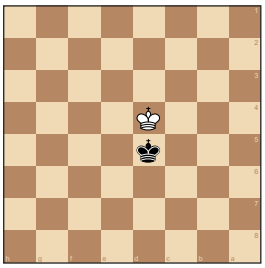
```
#board(
  "8/8/8/3qk3/3QK3/8/8/8",
  light: rgb("#eeeed2"),
  dark: rgb("#769656"),
  size: 3.8cm,
)
```



2.7 Flip

`flip: true` shows the board from Black's side; labels, highlights and arrows all flip with it. Orientation is a **per-board** choice — `flip` is the one setting that cannot be made a document default (see **Document-wide style**).

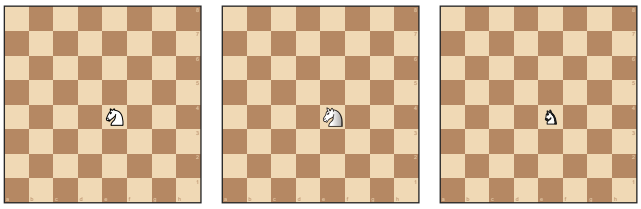
```
#board("8/8/8/3k4/3K4/8/8/8", flip:
true, size: 3.4cm)
```



2.8 Piece sets and fonts

Pieces are drawn from bundled **SVG piece sets**. Two ship with the package, both GPLv2+: `cburnett` (default) and `merida`. Choose one with `piece-set:` per board, or document-wide with `set-piece-set`:

```
#grid(columns: 3, gutter: 8pt, align: bottom,
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "cburnett"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "merida"),
  board("8/8/8/8/4N3/8/8/8", size: 2.6cm, piece-set: "unicode"),
)
```



The renderer accepts **any** set name and loads `src/assets/piece_sets/<name>/{w,b}{K,Q,R,B,N,P}.svg` on demand, so you can add a set by dropping such a folder into your copy and passing its name. `piece-set:` `"unicode"` (or `none`) selects the glyph fallback — solid Unicode chess glyphs distinguished by fill and a contrasting stroke; it needs a font carrying them.

The rank/file **labels** are drawn in their own sans-serif, set by the `label-font` board option (a family or a fallback list), independent of the document font. The default is `("Arial", "DejaVu Sans Mono")` — Arial on Windows/macOS, falling back to Typst's always-embedded mono — so a stock install draws labels without "unknown font family" warnings. Override it like any board default:

```
#set-board-defaults(label-font: "Segoe UI")   // or a list, e.g. ("Helvetica",
"DejaVu Sans Mono")
```

3 Diagrams

`chess-diagram(source, ..)` wraps a board in a `#figure` (kind `"chess"`), so — unlike a bare `board` — it is captioned, counted, referenceable, and listed by an outline. `source` is the same FEN / position / squares the board takes, and it accepts every board option from the previous chapter.

A diagram carries two optional labels: a **game-info** line above the board (drawn automatically as `"<White> – <Black> (<Year>)"` when both players are known) and the figure **caption** below it.

```
#chess-diagram(
  "r1bqkbnr/pppp1ppp/2n5/4p3/4P3/5N2/
PPPP1PPP/RNBQKB1R",
  white: "Morphy", black: "NN", year:
1858,
  caption: [After 2...Nc6],
  size: 4.2cm,
)
```

Morphy – NN (1858)

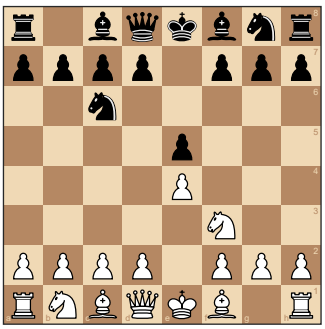


Diagram 2: After 2...Nc6

`chess-diagram` is the standard-chess sugar over the generic `diagram`; both take the same source and overrides as `board`. Extra named arguments are forwarded to `figure` (e.g. `placement: top`).

3.1 Boards are not figures

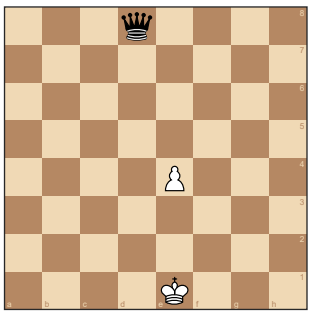
A bare `board` is plain content: it has **no** caption, **no** figure counter, does **not** resolve `@`-references, and is **not** listed by `chess-diagram-outline`. Only a **diagram** is a figure. So draw a `board` for an inline or

decorative position, and a `chess-diagram` whenever you want to caption it, cross-reference it (`@label`), or list it — see **Outlines and references**.

4 Positions

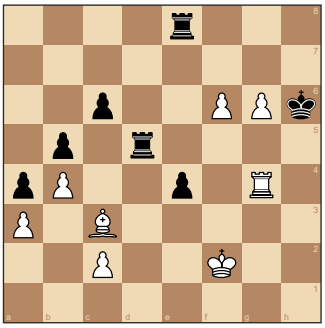
`position(..)` builds the data model — "which piece stands on which square" — that a board or diagram draws. It accepts a FEN string (auto-detected), a **squares dict**, or a **string form**. In a squares dict the piece can be a long name, a kind abbreviation, or a bare letter (UPPER = white, lower = black):

```
#chess-diagram(
  position((
    e1: (kind: "king", color: "white"), // long
    d8: (kind: "q", color: "black"),    // kind
    e4: "P",                            // letter
  )),
  size: 4cm,
)
```



The **string form** reads like the board itself — first line is the TOP rank, `.` is empty. Pass it as a raw block, as below — the most legible, least error-prone way, with no per-line quotes or commas (several row strings work too). It is rectangular-only (this is what lets a board be non-8×8) and rejects characters that aren't a valid piece abbreviation or `.`:

```
#chess-diagram(
  position(```
    ....r...
    ........
    ..p..PPk
    .p.r....
    pP..p.R.
    P.B.....
    ..P..K..
    ........
  ```),
  size: 4.2cm,
)
```



`position` returns a dict `(variant, cols, rows, squares, turn, castling, en-passant, halfmove, fullmove)`; `parse-fen` returns the same shape. The `cols` / `rows` are counted from the string form (otherwise the 8×8 default).

## 5 Games (PGN)

THe predominant form of the distribution and publications of chess games (atleast for western chess) are *PGN files*. PGN stands for **Portable Game Notation** and is a text format for chess games. It is human-readable, and can be parsed by chess software. PGN files contain the moves of a game, along with metadata such as player names, event, date, and result. PGN files cao contain just one or many games.

The `parse-pgn` function returns an **array of games**; `.first()` takes the first one. Read an external file with `read` in your own file, or pass an inline raw block. The examples below assume an already parsed game is in scope:

```
#let game = parse-pgn(read("game.pgn")).first()
```
```

Parsing is **lazy**: the roster (`tags`), the `result` , and the verbatim `movetext-raw` are extracted cheaply; the move tree is built on demand by `movetext(game)` ; the move parser / generator engine runs only when you ask for a position. So a tournament file read only for results and never tokenises movetext.

`chess-notation(game)` renders the moves (as text) the game already holds, and `board-after(game, loc)` draws the position at a locator:

```
#chess-notation(game)
```

1. e4 e5 2. Nf3 Nc6 3. Bb5 a6 4. Ba4 Nf6 5. O-O Be7 6. Re1 b5 7. Bb3 d6 8. c3 O-O

```
#board-after(game, "3w", size: 4cm)
```

Alice – Bob (2024)

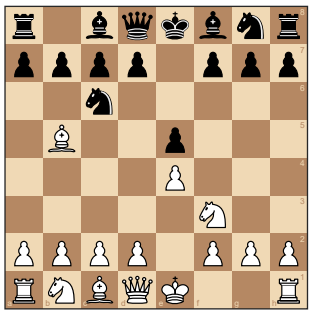


Diagram 5: Position after move 3. Bb5

5.1 Locators

A **locator** addresses one position in a game. The simple form is a string — `"30w"` / `"30b"` , the position after White's / Black's 30th **mainline** move. `position-after(game, loc)` , `board-after(game, loc, ..)` , and `to-fen(game, locator: ..)` all take this simple string form.

To address a move **inside a variation** (a PGN 'Recursive Annotation Variantion' or RAV), pass a **path** dict instead: `(line: (..hops..), at: "<final move>")` . Each hop is `(at: "<move>", into: <n>)` — branch off at mainline move `at` , descending `into` that move's variation number `n` (**0-based**: `0` is the first variation recorded at that move, `1` the second, ...). The top-level `at` is where you stop within the line you reached:

```
#let g = parse-pgn(
  "[White \"V\"][Black \"T\"] 1. e4 (1.
d4 d5 2. c4) e5 *",
).first()
// into variation 0 at White's move 1
(the
// 1.d4 line), position after 2.c4:
#board-after(
  g,
  (line: ((at: "1w", into: 0),), at:
"2w"),
  size: 3.4cm,
)
```

V – T

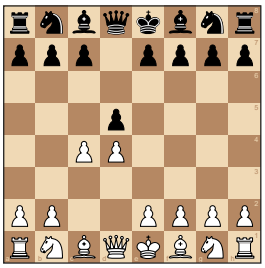


Diagram 6: Position after move 2. c4

Two easy traps:

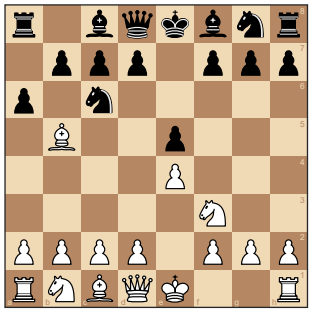
- **The trailing comma.** A single-hop path is written `((at: "1w", into: 0),)` — the comma makes it a one-element **array** of hops. Without it, Typst reads a lone parenthesised dict and the locator is malformed.
- **Mainline is the fast path.** The string form (equivalently an empty `line: ()`) indexes a memoised list of mainline positions, so many diagrams off one game stay cheap; the path form is walked move by move.

Addressing a variation that isn't there (`into:` past the recorded count) or a move past the end of its line is a hard error.

5.2 Playing moves onto a position

To explore a **new** line, or build a position from a FEN plus some moves, use `play-moves(source, moves)` . `source` is `none` (the standard start), a FEN string, or a position; `moves` is move text or a SAN array. It resolves each move against the legal moves (illegal/ambiguous is a hard error) and returns the **final** position, never mutating the source:

```
#chess-diagram(
  play-moves(none, "1. e4 e5 2. Nf3 Nc6
3. Bb5 a6"),
  size: 4cm,
)
```



5.3 Notation output

For chess publications, notational output of the move text is as important as showing board positions. This output has to be flexible and localisable. While you sometimes want to show move text exactly as it was recorded in the PGN, you often want to reformat the move text for your own purposes. The `notation(..)` function is the workhorse for this. It takes a game, a move-text string, or a SAN array and produces a formatted move text output. It can localise the piece letters and render figurine glyphs, and it can include or exclude move numbers, results, NAGs, comments, and embedded diagrams.

Two names, one formatter. `notation(..)` is the **variant-agnostic** primitive; `chess-notation(..)` is the **standard-western-chess** wrapper over it — identical output today, but `chess-notation` fixes the variant and rejects a source of a non-standard `variant` . The split is deliberate and forward-looking: staunton is **variant-forward**, so a future variant gets its own name — a planned `xiangqi-notation` , `shogi-notation` , … — each a thin wrapper over the same generic `notation` core, while `chess-notation` stays western-chess-specific. The same pairing runs through the whole package (`board` / `chess-board` , `diagram` / `chess-diagram` , `position` , `play-moves`): reach for the `chess-` name for ordinary chess, and the generic one only when you are deliberately variant-agnostic. Both this manual and everyday use favour `chess-notation` .

Either formats move text the game already holds — no engine. Its `source` is a game, a move-text string, or a SAN array; it localises the piece letters and renders figurine glyphs:

```
#chess-notation(game, lang: "de")
```

1. e4 e5 2. Sf3 Sc6 3. Lb5 a6 4. La4 Sf6 5. O-O Le7 6. Te1 b5 7. Lb3 d6 8. c3 O-O

```
#chess-notation(game, figurine: true)
```

1. e4 e5 2. ♘f3 ♞c6 3. ♗b5 a6 4. ♗a4 ♞f6 5. O-O ♝e7 6. ♖e1 b5 7. ♗b3 d6 8. c3 O-O

`from` / `to` restrict output to an **inclusive** slice of moves. Both are the simple `"8b"` / `"12w"` locators; `from` defaults to the first move, `to` to the last:

```
#chess-notation(game, from: "2w", to: "3b")
```

2. Nf3 Nc6 3. Bb5 a6

`from` / `to` bound a slice of the **mainline**: they are the simple `"8b"` / `"12w"` locators, not the variation **path** form that `board-after` takes (rendering variations is a separate control — see the next section). A `from` past the end or a `to` before `from` is a hard error.

Other options: `move-numbers`, `result`, and — for a **game** source — `nags` / `comments` (consulting the PGN-handling defaults). Localization substitutes only the piece letters; files, ranks, captures, check marks and `O-O` are untouched.

5.3.1 Variations

By default `notation` renders only the **mainline**. Set `variations: true` (or the `set-pgn-defaults(variations: ..)` document default) to splice the game's variations (RAVs) into the output — in parentheses, correctly numbered: a white-first line reads `N. …`, a black-first line `N… …`, and the resumed mainline move re-shows its number:

```
#chess-notation(
  parse-pgn(
    "1. e4 e5 2. Nf3 Nc6 3. Bb5 (3. Bc4 Bc5) a6 *",
  ).first(),
  variations: true,
)
```

1. e4 e5 2. Nf3 Nc6 3. Bb5 (3. Bc4 Bc5) 3... a6

Variations nest to any depth and honour `nags` / `comments` / `figurine` / `lang` inside the parentheses. `variation-style: "block"` instead breaks each variation onto its own line, indented one level per nesting depth — the analysis-view layout:

```
#chess-notation(
  parse-pgn(
    "1. e4 (1. d4 d5 (1... Nf6 2. c4)) e5 *",
  ).first(),
  variations: true,
  variation-style: "block",
)
```

1. e4
1. d4 d5
1... Nf6 2. c4
1... e5

To render **one specific** variation on its own, pass `line:` — a path locator (the same `board-after` shape, or just its hops array) that descends into the variation. It is numbered from its real branch ply, so it reads exactly as you'd write it in analysis:

```
#let g = parse-pgn(
  "1. e4 e5 2. Nf3 Nc6 (2... d6 3. d4) 3. Bb5 *",
).first()
// the 2...d6 side line, on its own:
#chess-notation(g, line: ((at: "2b", into: 0),))
```

2... d6 3. d4

Each hop is `(at: "<move>", into: <n>)` — nest hops to reach a deeper line. The addressed line's **own** variations follow the `variations` flag. (`from` / `to` stay mainline-only and don't combine with `line`.)

5.3.2 Programmatic NAGs

`nags: true` renders the NAGs a game already carries (the `$n` glyphs in the PGN). To attach NAGs **without editing the PGN**, use `with-nags(game, map)`: it returns a **new** game whose addressed mainline moves carry the given NAGs, which `notation(.., nags: true)` then renders.

```
#let g = parse-pgn(
  "[White \"A\"][Black \"B\"] 1. e4 e5 2. Nf3 Nc6 *",
).first()
#chess-notation(
  with-nags(g, ("2w": "!", "2b": "$14")),
  nags: true,
)
```

1. e4 e5 2. Nf3! Nc6±

Each map key is a **mainline** locator (`"2w"` / `"2b"`); each value is a `"$n"` code, one of the six suffix glyphs `! ? !! ?? !? ?!` (sugar for `$1` – `$6`), or an array of those. The mapping **replaces** any NAG already on that move, and the source game is never mutated. Only mainline moves are addressable — `notation` renders the mainline only.

5.4 Exporting FEN

`to-fen` is the inverse of `parse-fen`. It serialises a position, or a game at a locator. Standard 8×8 positions round-trip exactly:

```
#raw(to-fen(play-moves(none, "1. e4 e5 2. Nf3")))
```

```
rnbqkbnr/pppp1ppp/8/4p3/4P3/5N2/PPPP1PPP/RNBQKB1R b KQkq - 1 2
```

5.5 Drawing annotations

PGN comments can carry drawing annotations — `[%cal …]` for arrows, `[%csl …]` for highlights. Processing is **off by default**; opt in per call with `annotations: true` (or document-wide with `set-pgn-defaults`). The demo game annotates its 2nd move:

```
// move 2: {[%cal Gf3e5] [%csl Re5]}
#board-after(game, "2w", annotations:
true, size: 4cm)
```

Alice – Bob (2024)

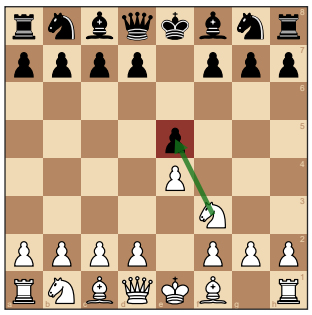


Diagram 8: Position after move 2. Nf3

The color letters (`G R Y B O`) resolve through the `annotation-colors` board style; annotations merge with any `arrows` / `highlight` you pass explicitly.

With embedded diagrams. The `diagrams` switch (PGN handling) makes `notation(diagrams: true)` splice a board after each move whose comment carries a diagram marker. If that **same** comment also holds `%cal` / `%csl` **and** `annotations` is on, the spliced board shows those arrows/highlights too — the two switches compose:

```
// 2. Nf3 {[%cal Gf1c4] [%csl Re5] #[After 2.Nf3]} Nc6 ...
#set-pgn-defaults(diagrams: true, annotations: true)
#chess-notation(game)   // ...text, then a board after 2.Nf3 with the arrow +
highlight
```

`diagrams` decides **whether** a board appears; `annotations` decides whether it carries the marks. The marker and the `%cal` / `%csl` must be in the **same** move's comment. (Only mainline moves are addressed — `notation` renders the mainline.)

5.6 Errors

Malformed PGN is a **hard error**: broken tag syntax and stray variation parens fail at parse time; illegal, ambiguous, or unparseable moves fail when the position is navigated. Missing Seven-Tag-Roster tags are tolerated (they default).

6 Tournament tables

Tournament tables are built from a parsed PGN's roster + results (no engine). The `*-table` renderers produce a captioned, referenceable `#figure` (kind `"chess-table"`). Each works on a list of games, with `by: "player"` or `by: "team"` . The examples use a parsed 4-player round-robin in `games` :

```
#let games = parse-pgn(read("event.pgn"))
```

`standings-table` sorts best-first (score, then tie-breaks, then first appearance):

```
#standings-table(games, by: "player")
```

Pos	Player	Pl	+	=	−	Pts	Bh	SB
1	Alice	3	2	1	0	2½	3½	2½
2	Dave	3	1	2	0	2	4	2½
3	Bob	3	1	0	2	1	5	½
4	Carol	3	0	1	2	½	5½	1

`crosstable-table` renders the round-robin grid (it **requires** a complete round-robin and errors otherwise — use standings + progress for Swiss/league):

```
#crosstable-table(games, by: "player")
```

#	Player	1	2	3	4	Pts
1	1 Alice		½	1	1	**2½**
2	2 Dave	½		1	½	**2**
3	3 Bob	0	0		1	**1**
4	4 Carol	0	½	0		**½**

`progress-table` shows the round-by-round running score (needs the `Round` tag):

```
#progress-table(games, by: "player")
```

#	Player	R1	R2	R3	Pts
1	Alice	1 (1)	1 (2)	½ (2½)	**2½**
2	Dave	½ (½)	1 (1½)	½ (2)	**2**
3	Bob	0 (0)	0 (0)	1 (1)	**1**
4	Carol	½ (½)	0 (½)	0 (½)	**½**

Each renderer takes `caption` (used by refs and the outline), `title` (a heading above the table), `supplement`, and `lang`. The compute functions (`standings`, `crosstable`, `progress`) are also public, returning plain data for custom layouts. **Team** mode groups games by the `Round = "round.board"` convention into matches.

7 Outlines and references

Diagrams and tables each carry a distinct figure `kind` (`"chess"` / `"chess-table"`), so they get their own counters, can be **referenced** (`@label` → "Diagram 3" / "Table 2"), and can be **listed separately**:

```
#chess-diagram-outline()      // list of chess diagrams
#chess-table-outline()        // list of tournament tables
#chess-outlines()             // both, diagrams then tables

#chess-diagram(starting-fen, caption: [Start]) <start>
As shown in @start, ...
```

Only figures that carry a **caption** appear in an outline (a caption-less figure is still referenceable but unlisted, matching Typst). Titles are language-aware by default and settable per call (`title:`, `lang:`) or document-wide (`set-diagram-defaults(outline-title: ..)`). Remember that a bare `board` is not a figure, so it can be neither referenced nor listed — use a `chess-diagram`.

8 Document-wide style

Defaults live in **five** buckets, each with its own setter:

bucket	setter	controls
board	`set-board-defaults`	square colours, labels, piece set, highlights, arrows, grid, size
diagram	`set-diagram-defaults`	the diagram figure: game-info line, supplement, outline title
table	`set-table-defaults`	the table figure: supplement, outline title, title gap

bucket	setter	controls
language	`set-lang`	the document language (also `set-i18n-defaults`)
PGN handling	`set-pgn-defaults`	`annotations` / `nags` / `comments` / `diagrams`

`set-chess-defaults` is the single **umbrella** over **all five** buckets: pass any key from any bucket — **including** `lang` — and it is routed to the bucket that owns it.

```
#set-board-defaults(light: rgb("#eeeed2"), dark: rgb("#769656"), size: 5cm)
#set-lang("de")
#set-pgn-defaults(nags: true)
// ...or all of the above at once, through the umbrella:
#set-chess-defaults(dark: blue, lang: "de", nags: true, info-gap: 1em)
#set-piece-set("merida")    // sugar for set-board-defaults(piece-set: ..)
```

A setter affects **every subsequent** diagram/table; per-call arguments still override. Every default is equivalently a per-call argument — the same green theme, set once above vs. passed to one diagram:

```
#chess-diagram(
  "rnbqkbnr/pppppppp/8/8/8/8/PPPPPPPP/
RNBQKBNR",
  light: rgb("#eeeed2"), dark:
rgb("#769656"),
  size: 4cm,
)
```

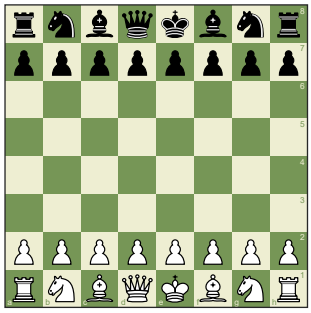


Diagram 9: Position at move 1, White to play

Two caveats. `flip` is **not** allowed in any defaults setter — orientation is a per-board choice, so `set-chess-defaults(flip: ..)` is an error. And `supplement` / `outline-title` live in **both** the diagram and table buckets; the umbrella routes them to **diagram**, so use `set-table-defaults` for the table ones.

8.1 Language

A single document **language** drives every language-aware string — diagram and table supplements, outline titles, and notation piece letters. Default is English; `"auto"` follows `#set text(lang: ..)`; or pick a code:

```
#set-lang("de")     // Diagramm / Tabelle / Sf3, Lb5, ...
#set-lang("auto")   // follow #set text(lang: ..)
```

Every localizable string is also per-call overridable (the `lang:` argument seen on `chess-notation` above). Adding a language is a no-code change: drop a `src/assets/i18n/<code>.typ` and register it in `src/i18n.typ`.

8.2 PGN handling

A **PGN-handling** bucket decides how much of a parsed game's embedded extras get interpreted at render time. Parsing stays lossless; these switches only decide what is processed, and **all default off**:

```
#set-pgn-defaults(annotations: true, nags: true, comments: true)
```

key	effect
`annotations`	`%cal` / `%csl` → arrows/highlights on `board-after`
`nags`	render NAGs (`Nf3!`, `d4±`) in `notation`

key	effect
`comments`	include comment prose in `notation`
`diagrams`	embed a board in `notation` after each move marked for one
`variations`	splice variations (RAVs) into `notation`, in parentheses

Each is also a per-call argument (`auto` ⟶ the document default) on `notation` / `board-after` — as used in the annotations example above.